

PDF report generator

Query your data with SQL, render it into a real PDF report, and let your API generate and hand out the file by link — no background jobs required.

~4–5 h across 7 stages

Stretch +2 h

Bonus AI stage +1 h

JavaScript or Python

\$0 · no credit card

PAIRED LIVE EVENT

Workshop — From data to inbox: automated PDF & email reports

YOU WILL PRACTICE

SQL aggregation · HTML-to-PDF rendering · Artifact handling · Serving files by link · Idempotency

SUBMISSION

Public GitHub repo, ≥ 7 commits, README

How to read this document: new words are shown in **bold** the first time they appear — every one of them is explained in the [Glossary](#) at the end. Work the stages **in order**; each ends with a checkpoint that proves it works. Background jobs are **not required** — if you did A7, a stretch goal shows where they slot in.

Contents

1. Goal & purpose	1	6. Requirements & stretch goals	7
2. The big idea in 60 seconds	2	7. Done means	8
3. Tools — pick one lane	2	8. Curated resources	9
4. The task — seven stages	2	9. Glossary	12
5. Bonus stage — the AI rematch	7		

1 · Goal & purpose

Goal: build the workshop's report pipeline yourself: put some data in a small **database**, shape it with one **SQL** query, **render** it into a real **PDF** report, and let your API generate and hand out that file — the client asks, then downloads the finished report by link. You publish everything to **GitHub**.

"Generate a report" is the most classic feature in software — every SaaS ships it. In the workshop you watched the pipeline get built: query → render → store → serve. Now you build it yourself, and it connects several skills in one feature: **SQL aggregation** turns rows into numbers, an **HTML template** turns numbers into a page, and a **headless browser** turns the page into a PDF. It also demos beautifully — at the end you hold a real file, made from your real data, by code you wrote.

You do not need background jobs for this assignment. The whole pipeline runs inside a plain endpoint, and that is exactly how you should build it here. In the workshop the pipeline was wrapped in a **background job** — and in Stage 4 you will *feel* why: your endpoint will take a few seconds to answer. That wait is precisely the problem A7 taught you to solve. If you did A7, a stretch goal shows you where its pattern slots in. If you didn't, nothing in the required stages depends on it.

This assignment also stands alone on data. You will create your own small dataset in Stage 1 — or, if you did **A9 — The polite scraper**, you can reuse the book data you collected from books.toscrape.com and report on that instead.

Beginners usually overthink the PDF part. You do not draw a PDF — you write a small HTML page and ask a browser to "print" it. The whole assignment is under 150 lines. You never write more than ~15 before you can test again.

2 · The big idea in 60 seconds

A report feature is a pipeline of four small moves:

Move	What it does	Where you build it
Query	One SQL query turns 200 rows into 5 numbers	Stage 2
Render	An HTML template + those numbers → a PDF file	Stage 3
Store	Save the file to disk; save its <i>path</i> in the database	Stage 4
Serve	The client downloads it by link — bytes stay on disk	Stage 4

That pipeline **is** the assignment. Stages 0–1 set the table, and Stage 5 makes sure asking twice doesn't make two files.

One rule carries the whole thing: **store and link, don't pass the bytes around**. A PDF is an **artifact** — a produced file. Artifacts live on disk (or in cloud storage later); everything else only ever holds their address.

3 · Tools — pick ONE lane

Both lanes build exactly the same thing. Pick the language you want to stick with; don't switch mid-assignment.

	JavaScript lane	Python lane
Language	Node.js 22+ (free, nodejs.org)	Python 3.10+ (free, python.org)
Web framework	Express — you know it from A1	FastAPI — you know it from A1
Database	SQLite via the built-in <code>node:sqlite</code> module	SQLite via the built-in <code>sqlite3</code> module
PDF rendering	Playwright — install , then <code>page.pdf()</code>	Playwright — install , then <code>page.pdf()</code>
Testing	<code>curl</code> + browser + a PDF viewer	
Publishing	Git + a free GitHub account	

SQLite is a real SQL database that lives in one file — no server to install, and it is built into both languages. Playwright is a tool that drives a real browser from code; its PDF printing works in headless Chromium, which it downloads for you (free, ~1 min). Everything in this table is \$0, no credit card.

4 · The task — seven stages (+ one bonus)

Work stage by stage, **in order**. Each stage ends with a checkpoint: a command you run, or a file you open, that proves it works. Commit to Git after every stage (that's your ≥ 7 commits, honestly earned). If you only finish Stage 3, submit anyway — **a working half is worth more than a broken whole**.

0 The setup

~30 min

A server and a printer. That's the whole workshop bench.

1. Create a small server with one endpoint: `GET /health → { "status": "ok" }`. You built this in A1 — a fresh copy of that first stage is all you need here.
2. Install Playwright and its browser: `npm install playwright` then `npx playwright install chromium` · `pip install playwright` then `playwright install chromium`.

CHECKPOINT — `curl -i http://localhost:3000/health` (or `:8000/health`) returns `200`, and `playwright install chromium` finished without errors.

Commit: `Stage 0: setup ready`

1 Data worth reporting on

~45 min

A report about nothing is nothing. Pick your dataset.

Pick **one** of the two options — the rest of the assignment works with either.

Option A — the little shop (works for everyone, no prerequisites):

1. Create a SQLite database `report.db` with one table `orders`: `id`, `customer` (text), `product` (text), `amount` (number), `created_at` (date). Use your lane's built-in module — no install needed.
2. Write a small script `seed` that inserts ~200 random orders: pick from 5–6 product names, random amounts between 5 and 200, random dates in the last 30 days. **Seed data** — invented rows for development — is a normal, professional tool.

Option B — the bookstore (if you did A9 — The polite scraper):

1. You already own a real dataset: the 60 validated book records your scraper collected from `books.toscrape.com` into `books.json`. Reuse them.
2. Create `report.db` with one table `books`: `id`, `title`, `price` (number), `rating` (number), `url`. Write a small script `seed` that reads your `books.json` and inserts the records.

Either way, one rule: run the seed script twice — and notice your row count doubled. Make the script **start by deleting all rows**, so running it twice leaves exactly one clean copy. (A tiny taste of "safe to run twice" — Stage 5 makes it a theme.)

CHECKPOINT — a query prints the row count: `SELECT COUNT(*) → 200` orders (or `60` books), even after seeding twice.

Commit: `Stage 1: seeded report.db`

2 Boring SQL is 80% of reporting

~1h

Nobody reads 200 rows. Everybody reads five numbers.

1. Write the queries that turn your rows into a report — this is **aggregation**: `COUNT`, `SUM`, `AVG`, `GROUP BY`. Your report needs four things. Shop: total number of orders · total revenue (`SUM(amount)`) · top 5 products by revenue (`GROUP BY product ORDER BY ... LIMIT 5`) · orders per day for the last 7 days. Bookstore: total number of books · average price (`AVG(price)`) · top 5 most expensive books (`ORDER BY price DESC LIMIT 5`) · number of books per star rating (`GROUP BY rating`).
2. Put them in one function `getReportData()` that returns a single object/dict with those four results.
3. Print it as JSON from a tiny test script. Look at the numbers. Do they make sense against your data? (If one product's revenue is bigger than total revenue, the bug is in the query, not the data.)

If `GROUP BY` is new or rusty: [SQLBolt lessons 10–11](#) rebuild it in ~20 minutes, in the browser.

CHECKPOINT — your test script prints the full report object: four sections, real numbers, valid JSON.

Commit: `Stage 2: aggregation queries`

3 Render: from numbers to a PDF

~1.5h

You do not draw a PDF. You write a web page and ask a browser to print it.

1. Write a function that builds an HTML page (a template string is fine) from your report object: a title with today's date, the two totals, a small table for the top 5, and a long table at the bottom — all orders, or all 60 books (long on purpose — you'll see why).
2. Render it with Playwright: launch headless Chromium, `setContent(html)`, then `page.pdf({ path, format: "A4", printBackground: true })`. Save to `reports/test.pdf`. (PDF printing works in Chromium — that's why Stage 0 installed it.)
3. Open the file. Now find the classic trap: the long table crosses a **page break**, and somewhere a row is probably sliced in half. Fix it with print CSS: `tr { break-inside: avoid; }`, and put the header row in a `<thead>` so the browser repeats it on every page.

Stop and notice: **your data just became a document**. Query → render is the heart of every invoice, statement, and export you have ever downloaded.

CHECKPOINT — `reports/test.pdf` opens, is at least 2 pages, no row is cut in half, and the table header appears on both pages.

Commit: `Stage 3: HTML to PDF with clean page breaks`

4 Serve it from your API

~45 min

The pipeline works by hand. Now put a door in front of it.

1. Add a `reports` table to `report.db: id, path, created_at`. Your report bookkeeping lives next to your data.
2. Add `POST /reports`: it runs the whole pipeline right there in the endpoint — query, render to `reports/<id>.pdf`, insert the row — and returns `201` with `{ "id": ..., "file": "/reports/<id>/file" }`. Yes, the response takes a few seconds. Here, that is allowed.
3. Add `GET /reports/:id` → the row, including the file `link`. Unknown id → `404`.
4. Add `GET /reports/:id/file` → **serves** the PDF from disk (`res.sendFile(...)` · `FileResponse(...)`). The other endpoints answer with a few bytes of JSON; only this one moves the megabytes. That is "store and link" in practice.
5. **Feel the wait.** Run the checkpoint and watch the POST hang for a few seconds before answering. For one user clicking one button, this is fine. For big reports or many users, it is not — a request that works for seconds is fragile and keeps the user hostage. That is exactly the problem background jobs solve, and if you did A7, the stretch shows how to apply it here. Write one sentence in your README: at what point would you move this work out of the request?

CHECKPOINT — `time curl -i -X POST http://localhost:3000/reports` returns `201` + a link after a visible pause; `curl -o my-report.pdf .../reports/<id>/file` downloads a file that opens as a real PDF.

Commit: Stage 4: generate and serve by link

5 Ask twice, get one

~30 min

A user double-clicks the "Generate report" button. Every user, every time, forever.

1. Make `POST /reports` check first: if a report was **already generated today**, do not generate again — return the existing id and link (with `200` instead of `201`). This is **idempotency** in business terms: same request twice → one effect, one file. (Want a fresh one anyway? Accept `{ "force": true }` and skip the check.)
2. Prove it: fire two POSTs back-to-back (arrow-up, enter, enter). Both answers carry the **same id**, and `reports/` gains exactly **one** new file.
3. Write two sentences in your README: what your check protects against, and one real-world example where a missing check like this costs money (the workshop's "never email a customer twice" is a fine answer).

CHECKPOINT — two rapid POSTs → same id in both responses, one new file in `reports/`. With `force: true` → a new id.

Commit: Stage 5: duplicate requests make one report

6 Publish to GitHub

~45 min

Your work only counts when someone else can run it.

1. Put the code in a **public GitHub repo**. Assignments in this program are independent: a new repo is fine, and adding this to a repo you used for another assignment is also fine. If one repo holds several assignments, give each assignment its own clearly named folder (for example `pdf-report-generator/`) so anyone can find the right work quickly. The public repo link is what you submit. Add `reports/` and `report.db` to `.gitignore` — generated artifacts and databases do not belong in Git; the seed script is their recipe.
2. Write a **README** with: what this is, which dataset you chose, how to run it (seed command + API command), your **pasted aggregation SQL**, the POST → download proof, your Stage 4 and Stage 5 sentences, and a screenshot of page 1 of a generated PDF.

CHECKPOINT — a stranger with your README could seed, run, and download a report in under 5 minutes.

Commit: `Stage 6: publish and docs` — then push everything.

★ Make it yours — optional extras

optional · as many or as few as you like

It's your report. Decorate.

None of these are required. Pick whatever sounds fun (creative alternatives welcome):

- **Make it pretty:** a logo, brand colors, a repeating footer with page numbers — print CSS can do all of it, and pretty demos win.
- **A parameterized report:** `POST /reports` accepts `{ "days": 7 }` (shop) or `{ "min_rating": 4 }` (bookstore) and the query respects it — one **query parameter** away from a real product feature.
- **The control panel:** `GET /reports` lists all generated reports with links.
- **Nice filenames:** `sales-report-2026-08-17.pdf` — small touch, very professional.
- **The big-table experiment:** seed 5,000 rows and generate again. Write two sentences in the README: how long did the endpoint take now, and what does that say about doing this inside a request?

Commit (if you build any): `Extras: <what you added>`

5 · Bonus stage — the AI rematch

7 The AI rematch

~1 h · optional — and the most fun

You built a data-to-document pipeline by hand. Now hire the fastest junior developer on Earth — and review their work.

You did Stages 0–6 by hand for a reason: you now know exactly what "correct" looks like. That knowledge turns this stage from a magic show into a code review.

1. **Write the prompt yourself — this is the real exercise.** Without copying text from this document, write your own **prompt** asking an AI assistant (Claude, ChatGPT, Gemini — any) to build the same feature. From memory, specify everything that matters: the schema and seed, the four aggregations, HTML-to-PDF with clean page breaks, the generate + status + file-link endpoints, the once-per-day rule. Describing a system precisely is a core backend skill.
2. **Generate in quarantine.** The AI's code goes in a separate folder (`ai-version/`) or branch. Your hand-built submission stays hand-built.
3. **Run it.** Fire your Stage 4 and Stage 5 checkpoints at it. Does its PDF survive the page-break trap?
4. **Diff it.** Compare side by side (`git diff --no-index`), then answer three questions in an **"AI vs me"** section of your README: What did the AI do *better* — and do you understand it? What did it get *wrong or silently ignore*? What did *your prompt* forget to specify — and what did the AI silently decide for you?
5. **One rematch.** Improve the prompt, regenerate, and note in one sentence what changed.

The lesson hiding in this stage: an AI's output is exactly as good as your specification — and you could only judge it because you built the thing yourself first. **Both halves of that sentence are your career from now on.**

CHECKPOINT — README has an "AI vs me" section with your full prompt and ≥ 3 concrete differences.

Commit: `Stage 7: AI vs me` (AI code stays in its own folder/branch).

6 · Requirements

Done = every box ticked. Each one is checkable in under a minute.

- ☐ `report.db` exists and the **seed script** fills it (shop: ~200 orders · bookstore: your 60 A9 books), even when run twice.
- ☐ One set of **aggregation queries** feeds the report (two totals, a top 5, a grouped breakdown) — pasted in the README.
- ☐ The **PDF renders** from your real data, is ≥ 2 pages, and no table row is cut by a **page break** (header repeats).
- ☐ `POST /reports` generates the report and returns `201` + an id and a file **link**; `GET /reports/:id` returns the record; unknown id → `404`.
- ☐ The file is **stored on disk and served by link** (`GET /reports/:id/file`) — JSON responses never carry the file's bytes.
- ☐ **Duplicate requests → one file:** two rapid POSTs on the same day return the same id and produce exactly one new PDF.
- ☐ Public **GitHub repo** (own repo, or a clearly named folder in a shared one), ≥ 7 meaningful commits, README with run commands, the pasted SQL, the download proof, your Stage 4 & 5 sentences, and the PDF screenshot. `reports/` and `report.db` are gitignored.

Stretch (optional)

- ☐ **Bring in A7:** move generation into a **background job** — `POST /reports` returns `202` + id instantly, an Inngest function runs query → render → save as separate steps, and `GET /reports/:id` reports `pending` / `done` . One README line: what got better for the user, and what got more complex for you?
- ☐ **Cron it:** generate the report **every Monday at 08:00** with no request at all (build the expression on crontab.guru) — an Inngest `scheduled function` if you did A7, or `node-cron` / `APScheduler` if not. One README line: what happens to Monday's report if the server was down at 08:00?
- ☐ **Email it:** run `Mailpit` (a free local **mailcatcher** with a web inbox), and after generating, "send" an email containing the report **link** — **not the file as an attachment**. One README line on why links beat attachments.
- ☐ The AI rematch (section 5): prompt an AI to build the same feature, run it, diff it, and write your "AI vs me" section.

7 • Done means

- The full ride works in one terminal session: seed → `POST /reports` (`201` + link after a visible pause) → download → a real PDF of your real data opens, ≥ 2 clean pages.
- The double-click proof: two rapid POSTs, same id, one new file — shown in the README.
- The repo is public, the README works on a clean machine, and `git log` shows one honest commit per stage.

8 · Curated resources

Don't read everything. Each section says *when* you need it. All resources are free, no credit card. Background jobs are **not required** — the Inngest rows serve the optional stretches.

● **Start here** (everyone) · ● **When you're comfortable** · ● **Optional deep dive** · **JS / Py** = lane-specific

§1 · The pipeline — see it whole first

Resource	Format	Why it's useful
● The workshop recording — From data to inbox (in the portal)	Video, ~60 min	This pipeline built live: query → render → store → send. Prerecorded — pause and build along. The workshop wraps it in a background job; in A8 that part is the stretch, not a requirement.
● Keeping your API fast (Inngest)	Article, ~5 min	Why slow work eventually leaves the request — the A7 idea, one page. Read after Stage 4's "feel the wait" moment.

§2 · SQLite & SQL — data and aggregation

Resource	Format	Why it's useful
● JS node:sqlite — Node.js docs	Reference, ~10 min	The built-in module: DatabaseSync, prepare, run, all. Everything Stages 1–2 need, no install.
● Py sqlite3 — Python docs	Reference + tutorial, ~15 min	The built-in module, with a tutorial section at the top. Everything Stages 1–2 need, no install.
● SQLBolt — lessons 10–11: queries with aggregates	Interactive, ~20 min	COUNT, SUM, GROUP BY, HAVING — rebuilt in your browser with instant feedback. Do before Stage 2 if GROUP BY is new or rusty.
● Books to Scrape + your A9 <code>books.json</code>	Sandbox + your data	Stage 1, Option B: the book data you scraped in A9 — The polite scraper is a ready-made real dataset. Load it and report on it.
● SQL basics guide — GROUP BY tier (program resources)	Guide	The program's own SQL ladder — the aggregate tier maps 1:1 onto Stage 2.
● SQLBolt — order of execution	Interactive, ~10 min	Why WHERE can't see aggregates but HAVING can. Read when a query error confuses you.

§3 · Playwright & PDF rendering

Resource	Format	Why it's useful
● JS Playwright installation	Docs, ~5 min	<code>npm install playwright + npx playwright install chromium</code> — Stage 0's second step.
● Py Playwright for Python — installation	Docs, ~5 min	<code>pip install playwright + playwright install chromium</code> — same, Python flavor.

● JS	page.pdf() reference	Reference	The options that matter: path, format: "A4", printBackground, margins. PDF printing works in headless Chromium.
● Py	page.pdf() — Python reference	Reference	The same call in Python. Look up, don't read.
●	Generate PDFs from HTML templates with Node.js	Article, ~15 min	A worked HTML-template-to-PDF example — close to Stage 3's shape. Read if you want a model to compare against.

§4 · The page-break trap — print CSS

Resource	Format	Why it's useful
● How to fix PDF page breaks in HTML	Article, ~10 min	<code>break-inside: avoid</code> , <code>break-before</code> , and friends — the Stage 3 fix, explained. Read <i>when the row gets sliced</i> , not before.
● Page breaks when printing large HTML tables (W3Docs)	Snippet, ~5 min	The thread-repeats-per-page trick for long tables, in one copy-pasteable example.
● Repeating table headers in PDF downloads	Article, ~10 min	Deeper dive into how headless Chrome decides what repeats — for when the simple trick isn't enough.

§5 · Serving files — and the optional job & cron stretches

Resource	Format	Why it's useful
● JS Express res.sendFile	Reference	Stage 4's download endpoint in one call.
● Py FastAPI FileResponse	Reference	The same, FastAPI flavor.
● Background jobs with Inngest — setup guide	Docs, ~10 min	For the "bring in A7" stretch only: the event → function shape that moves generation out of the request.
● Scheduled functions & cron + crontab.guru	Docs + tool	For the cron stretch, the A7 road: Mondays at 08:00 is <code>0 8 * * 1</code> — build it yourself and check.
● JS Schedule a job in Node with node-cron (freeCodeCamp)	Tutorial, ~10 min	For the cron stretch without Inngest: scheduling inside your own Node process.
● Py APScheduler	Docs	For the cron stretch without Inngest: Python's in-process scheduler with cron-style triggers.
● Mailpit	Tool	For the email stretch: a free local mailcatcher — your app "sends", Mailpit catches, you read it in a web inbox. Nothing leaves your machine.

Critical resources per stage

Stage	Keep these open
Stage 0	Your lane's Playwright install page (§3).
Stages 1–2	Your lane's SQLite module docs (§2) · SQLBolt lessons 10–11 · Option B: your A9 <code>books.json</code> .
Stage 3	The <code>page.pdf()</code> reference (§3) · the page-break guides (§4) — when the row gets sliced.
Stages 4–5	Your lane's file-serving reference (§5).

Suggested path for a complete beginner (~1.5 h before starting)

1. The workshop recording (§1) — ~60 min, or at least the query and render segments. The map.
2. [SQLBolt lessons 10–11](#) — ~20 min in the browser. The SQL.
3. Your lane's Playwright install page (§3) — ~5 min. The renderer.
4. Start Stage 0. Come back to §4–§5 only when a stage sends you here.

Everything above is free. If a link ever dies, the titles are searchable — the material outlives the URL.

9 · Glossary

Plain-language definitions of every **bold** word above. No definition depends on another — read them in any order.

Word	What it means
SQL	The language for asking questions of a database: SELECT, WHERE, GROUP BY. Fifty years old and running the world's reports daily.
Database	A program (or file) that stores data and answers SQL questions about it — here, a single file called <code>report.db</code> .
SQLite	A tiny, real SQL database that lives in one file on disk. Built into Python and modern Node — nothing to install or configure.
Seed data	Rows inserted by a script so development has something realistic to work with. The script is the recipe; the data is disposable.
Aggregation	Turning many rows into few numbers: COUNT, SUM, AVG. The difference between 200 orders and "total revenue: €4,310".
GROUP BY	The SQL clause that splits rows into buckets (per product, per rating, per day) so aggregation happens per bucket. The engine of every "top 5" list.
Template	A page with holes in it. You pour in this report's numbers, and out comes this report's HTML.
Render	Turning data into something viewable — here, numbers into HTML, then HTML into PDF pages.
PDF	The file format for documents that must look the same everywhere. Reports become PDFs because layouts freeze in place.
Headless browser	A real browser running without a window, driven by code. It loads your HTML and "prints" it to PDF exactly as Chrome would.
Playwright	The free tool that drives the headless browser from Node or Python: load a page, click, screenshot — and <code>page.pdf()</code> .
Page break	The invisible line where one PDF page ends and the next begins. Uncontrolled, it slices table rows in half; print CSS controls it.
Artifact	A produced file — a PDF, an export, a build. Artifacts are stored (disk, cloud storage later) and referenced by address, never passed around in memory.
Store and link	The artifact rule: save the file once, hand out its address. JSON responses stay tiny; only the download endpoint moves megabytes.
Serve (a file)	Answering a request with a file from disk — <code>res.sendFile / FileResponse</code> — instead of JSON.
Link	An address for a thing (<code>/reports/17/file</code>) instead of the thing itself.
Status code	The 3-digit number in every response saying how it went: <code>200</code> OK, <code>201</code> Created, <code>404</code> Not Found.
Background job	Work your server starts but does not finish inside the request — the A7 pattern. Not required here; the stretch shows where it would slot in once the wait gets too long.
Idempotency	Same request twice → one effect. Here: a double-clicked "generate" makes one PDF, not two. The workshop's "never email a customer twice" is the same rule with higher stakes.
Query parameter	Extra options after <code>?</code> in a URL, or extra fields in a request body, that tune a request — like <code>days: 7</code> .

Cron	The scheduling system that starts work at fixed times, described by a five-field expression like <code>0 8 * * 1</code> (Mondays, 08:00). A7 Stage 4 teaches it.
Mailcatcher	A fake mail server for development: it accepts your app's emails and shows them in a local web inbox instead of delivering them. Mailpit is one.
Attachment	A file glued into an email. Heavy, duplicated per recipient — which is why report emails carry links instead.
.gitignore	The list of files Git must not track — generated artifacts, databases, secrets. The recipe is committed; the products are not.
Git / GitHub / repo / commit	Git tracks versions of your code in a repository ; a commit is one saved step with a message; GitHub hosts your repo online so others can see it.
README	The front page of a repo: what this is, how to run it, how to use it.
Prompt	The instructions you give an AI assistant. In the bonus stage your prompt is a mini-specification — the more precisely it names behaviors, the closer the output lands.
Diff	A line-by-line comparison of two versions of code. <code>git diff</code> produces one; reading diffs is how professionals review each other's work.

FlyRank Internship · Backend Development Track · Week 4 · Assignment A8 — PDF report generator. All linked resources are free with no credit card required.